

Claim Loss Cause / Segment Insights — Notebook Code

All cells from claim_insights_notebook.ipynb, in order. Monospace font preserves exact indentation for copy/paste.

Claim Loss Cause / Segment Insights — Presentation-Ready

Run cells top to bottom. Only the **CONFIG** cell needs editing.

Every chart is:

styled consistently (clean, labeled, no clutter) so you can screenshot it straight into slides

automatically saved as a PNG to `chart\_exports/` at presentation resolution, so you don't have to screenshot at all — just drag the files into PowerPoint

every table is also saved as a `.csv` to `chart\_exports/tables/` for backup/appendix use

Each cut also prints a short **Key Callouts** block — plain-text bullets you can paste directly into slide notes or a summary slide.

Covers, for each of 4 business cuts (**Overall**, **Property-Catastrophe**, **Property-Non-Catastrophe**, **Non-Injury**):

1. FNOL to latest status — days distribution
2. Loss-cause day movement — per-cause day stats, first→last cause heatmaps (frequency & avg days), total journey-length distribution
3. Line of business breakdown
4. Does a loss-cause change trigger a segment change?
5. Contact method breakdown

Plus a final **cross-cut comparison** section for an overview/summary slide.

All numbers are shown as-is (counts, percentages, simple averages) — no statistical modeling.

Setup

[code cell 1]

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display

pd.set_option("display.max_columns", None)
pd.set_option("display.width", 150)

# ---- Presentation styling ----
PRIMARY = "#2E5C8A"      # main bar color
ACCENT = "#DA8A2D"       # secondary bar color
```

```
GOOD = "#4E8B5C"          # tertiary
PALETTE_SERIES = [PRIMARY, ACCENT, GOOD, "#8172B2", "#C44E52"]
```

```
plt.rcParams.update({
    "figure.dpi": 110,
    "savefig.dpi": 220,
    "font.size": 11,
    "axes.titlesize": 14,
    "axes.titleweight": "bold",
    "axes.labelsize": 11,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "axes.grid": True,
    "grid.alpha": 0.3,
    "font.family": "sans-serif",
})
```

Config — edit this

[code cell 2]

```
INPUT_CSV = "your_claims.csv"      # <-- change this to your file path
OUTPUT_DIR = "chart_exports"       # PNGs + backup CSVs land here, ready to drag into slides

CHART_DIR = os.path.join(OUTPUT_DIR, "charts")
TABLE_DIR = os.path.join(OUTPUT_DIR, "tables")
os.makedirs(CHART_DIR, exist_ok=True)
os.makedirs(TABLE_DIR, exist_ok=True)
```

Load data

[code cell 3]

```
df = pd.read_csv(INPUT_CSV, dtype=str)

for col in ["CAUSE_CHANGE_COUNT", "SEGMENT_TRANSFER_COUNT", "FNOL_EFF_DIFF"]:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors="coerce")

print(f"Loaded {len(df):,} claims, {df.shape[1]} columns")
df.head()
```

Define the 4 business cuts

[code cell 4]

```
cuts = {
    "Overall": df,
    "Property - Catastrophe": df[(df["NEW_SEGMENT"] == "Property") & (df["CAT_IND"] == "Y")],
    "Property - Non-Catastrophe": df[(df["NEW_SEGMENT"] == "Property") & (df["CAT_IND"] == "N")],
    "Non-Injury": df[df["NEW_SEGMENT"] == "Non-Injury"],
}

for name, d in cuts.items():
    print(f"{name:30s} {len(d):>8,} claims")
```

Save / styling helpers

[code cell 5]

```
def slug(text):
    return "".join(c.lower() if c.isalnum() else "_" for c in text).strip("_")

def save_fig(fig, cut_name, chart_name):
    path = os.path.join(CHART_DIR, f"{slug(cut_name)}_{chart_name}.png")
    fig.savefig(path, bbox_inches="tight")
    return path

def save_table(dframe, cut_name, table_name, index=False):
    path = os.path.join(TABLE_DIR, f"{slug(cut_name)}_{table_name}.csv")
    dframe.to_csv(path, index=index)
    return path

def styled_bar(ax, categories, values, title, ylabel="Claim Count", color=PRIMARY, value_fmt="{:,.0f}"):
    bars = ax.bar(categories, values, color=color, width=0.62)
    for bar, v in zip(bars, values):
        ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height(),
                value_fmt.format(v), ha="center", va="bottom", fontsize=10, fontweight="bold")
    ax.set_title(title)
    ax.set_ylabel(ylabel)
    ax.margins(y=0.15)
    ax.tick_params(axis="x", labelrotation=0)
```

```

ax.yaxis.grid(True, alpha=0.3)
ax.xaxis.grid(False)
return ax

```

Analysis functions

All five insights, defined once and reused for every cut.

[code cell 6]

```

FNOL_BINS = [-1, 1, 3, 7, 14, 30, np.inf]
FNOL_LABELS = ["0-1", "2-3", "4-7", "8-14", "15-30", "30+"]

DAYS_BINS = [-1, 5, 10, 20, 30, 50, np.inf]
DAYS_LABELS = ["0-5", "6-10", "11-20", "21-30", "31-50", "50+"]

def fnol_analysis(d):
    valid = d["FNOL_EFF_DIFF"].dropna()
    bucketed = pd.cut(valid, bins=FNOL_BINS, labels=FNOL_LABELS)
    counts = bucketed.value_counts().reindex(FNOL_LABELS).fillna(0).astype(int)
    pct = (counts / max(len(valid), 1) * 100).round(1)
    table = pd.DataFrame({"Days Bucket": FNOL_LABELS, "Claim Count": counts.values, "% of Claims": pct.values})
    summary = pd.DataFrame({
        "Metric": ["Min Days", "Median Days", "Average Days", "Max Days", "Claim Count"],
        "Value": [
            round(valid.min(), 1) if len(valid) else 0,
            round(valid.median(), 1) if len(valid) else 0,
            round(valid.mean(), 1) if len(valid) else 0,
            round(valid.max(), 1) if len(valid) else 0,
            len(valid),
        ],
    })
    return table, summary

def parse_cause_days(s):
    if pd.isna(s) or str(s).strip() == "":
        return []
    out = []
    for part in str(s).split(">"):
        part = part.strip()
        if not part or ":" not in part:

```

```

        continue
    cause, days = part.rsplit(":", 1)
    try:
        out.append((cause.strip(), int(days.strip())))
    except ValueError:
        continue
    return out

```

```

def cause_days_analysis(d):
    chains = d["CAUSE_DAYS_DISTRIBUTION"].apply(parse_cause_days)

    first_last = []
    per_cause_days = {}
    total_days_per_claim = []
    for chain in chains:
        if not chain:
            continue
        causes = [c for c, _ in chain]
        days_list = [dd for _, dd in chain]
        first_last.append((causes[0], causes[-1], sum(days_list)))
        total_days_per_claim.append(sum(days_list))
        for c, dd in chain:
            per_cause_days.setdefault(c, []).append(dd)

    fl_df = pd.DataFrame(first_last, columns=["First Cause", "Last Cause", "Total Days"])
    freq_heatmap = pd.crosstab(fl_df["First Cause"], fl_df["Last Cause"]) if len(fl_df) else pd.DataFrame()
    days_heatmap = (fl_df.pivot_table(index="First Cause", columns="Last Cause",
                                      values="Total Days", aggfunc="mean").round(1)
                    if len(fl_df) else pd.DataFrame())

    cause_rows = []
    for c, vals in per_cause_days.items():
        cause_rows.append({
            "Loss Cause": c,
            "Times Occurred": len(vals),
            "Total Days (sum)": int(sum(vals)),
            "Avg Days (when present)": round(float(np.mean(vals)), 1),
            "Median Days": round(float(np.median(vals)), 1),
        })
    cause_stats = pd.DataFrame(cause_rows).sort_values("Times Occurred", ascending=False).reset_index(drop=True)

    n_single = sum(1 for c in chains if len(c) == 1)

```

```

n_multi = sum(1 for c in chains if len(c) > 1)
n_total = max(n_single + n_multi, 1)
single_multi = pd.DataFrame({
    "Claim Type": ["Single Loss Cause", "Multiple Loss Causes"],
    "Claim Count": [n_single, n_multi],
    "% of Claims": [round(n_single / n_total * 100, 1), round(n_multi / n_total * 100, 1)],
})

total_days_series = pd.Series(total_days_per_claim, dtype=float)
bucketed = pd.cut(total_days_series, bins=DAYS_BINS, labels=DAYS_LABELS)
days_counts = bucketed.value_counts().reindex(DAYS_LABELS).fillna(0).astype(int)
days_pct = (days_counts / max(len(total_days_series), 1) * 100).round(1)
total_days_table = pd.DataFrame({
    "Total Journey Days": DAYS_LABELS, "Claim Count": days_counts.values, "% of Claims": days_pct.values
})

return {
    "freq_heatmap": freq_heatmap, "days_heatmap": days_heatmap, "cause_stats": cause_stats,
    "single_vs_multi": single_multi, "total_days_table": total_days_table,
    "total_days_series": total_days_series, "fl_df": fl_df,
}

def breakdown_table(d, col, label):
    counts = d[col].value_counts()
    pct = (counts / max(len(d), 1) * 100).round(1)
    return pd.DataFrame({label: counts.index, "Claim Count": counts.values, "% of Claims": pct.values})

def cause_to_segment(d):
    dd = d.copy()
    dd["Cause Changed?"] = np.where(dd["CAUSE_CHANGE_COUNT"] > 0, "Cause Changed (1+)", "No Cause Change")
    dd["Segment Changed?"] = np.where(dd["SEGMENT_TRANSFER_COUNT"] > 0, "Segment Changed (1+)", "No Segment Change")
    ct = pd.crosstab(dd["Cause Changed?"], dd["Segment Changed?"])
    ct_pct_row = ct.div(ct.sum(axis=1), axis=0).mul(100).round(1)
    cause_changed = dd[dd["Cause Changed?"] == "Cause Changed (1+)"]
    trigger_rate = (round((cause_changed["Segment Changed?"] == "Segment Changed (1+)").mean() * 100, 1)
                    if len(cause_changed) else 0.0)
    headline = pd.DataFrame({
        "Metric": ["Claims with a Cause Change", "...of those, % that ALSO had a Segment Change"],
        "Value": [len(cause_changed), trigger_rate],
    })
    return ct, ct_pct_row, headline, trigger_rate

```

Report function

Runs all 5 insights for one cut: prints Key Callouts, displays every table, builds every chart, and saves everything to `chart\_exports/`.

[code cell 7]

```
def run_report(name, d):
    print("=" * 95)
    print(f"{name} - {len(d):,} claims")
    print("=" * 95)

    if len(d) == 0:
        print("(no claims in this cut)")
        return {}

    results = {"cut": name, "n_claims": len(d)}

    # ----- 1. FNOL -----
    fnol_table, fnol_summary = fnol_analysis(d)
    print("\n--- 1. FNOL to Latest Status (FNOL_EFF_DIFF) ---")
    display(fnl_table); display(fnl_summary)
    save_table(fnl_table, name, "1_fnl_distribution")

    fig, ax = plt.subplots(figsize=(7.5, 4))
    styled_bar(ax, fnl_table["Days Bucket"], fnl_table["Claim Count"],
               f"{name}: FNOL to Latest Status - Days Distribution")
    save_fig(fig, name, "1_fnl_distribution")
    plt.show()
    results["avg_fnl_days"] = fnl_summary.loc[fnl_summary["Metric"] == "Average Days", "Value"].iloc[0]

    # ----- 2. Loss-cause day movement -----
    print("\n--- 2. Loss-Cause Day Movement (CAUSE_DAYS_DISTRIBUTION) ---")
    cd = cause_days_analysis(d)

    print("Single vs multiple loss-cause claims:")
    display(cd["single_vs_multi"])
    save_table(cd["single_vs_multi"], name, "2a_single_vs_multi_cause")

    print("Per-cause day stats:")
    display(cd["cause_stats"])
    save_table(cd["cause_stats"], name, "2b_per_cause_day_stats")
```

```

print("Total journey length per claim (sum of days across the full cause chain):")
display(cd["total_days_table"])
save_table(cd["total_days_table"], name, "2c_total_journey_days")

fig, ax = plt.subplots(figsize=(7.5, 4))
styled_bar(ax, cd["total_days_table"]["Total Journey Days"], cd["total_days_table"]["Claim Count"],
           f"{name}: Total Days in Loss-Cause Journey (per claim)", color=G00D)
save_fig(fig, name, "2c_total_journey_days")
plt.show()

if len(cd["freq_heatmap"]):
    fig, axes = plt.subplots(1, 2, figsize=(15.5, 6))
    sns.heatmap(cd["freq_heatmap"], annot=True, fmt="d", cmap="Blues", ax=axes[0],
               cbar_kws={"label": "Claim Count"}, linewidths=0.5, linecolor="white")
    axes[0].set_title(f"{name}: First → Last Cause – Frequency")
    sns.heatmap(cd["days_heatmap"], annot=True, fmt=".1f", cmap="Oranges", ax=axes[1],
               cbar_kws={"label": "Avg Days"}, linewidths=0.5, linecolor="white")
    axes[1].set_title(f"{name}: First → Last Cause – Avg Total Days")
    plt.tight_layout()
    save_fig(fig, name, "2d_first_last_cause_heatmaps")
    plt.show()
    save_table(cd["freq_heatmap"].reset_index(), name, "2d_first_last_cause_frequency", index=False)
    save_table(cd["days_heatmap"].reset_index(), name, "2d_first_last_cause_avg_days", index=False)
    results["top_first_cause"] = cd["cause_stats"].iloc[0]["Loss Cause"] if len(cd["cause_stats"]) else "n/a"
else:
    print("(not enough data to build the first->last cause heatmap for this cut)")

# ----- 3. Line of business -----
print("\n--- 3. Line of Business Breakdown ---")
lob = breakdown_table(d, "LINE_OF_BUSINESS", "Line of Business")
display(lob)
save_table(lob, name, "3_line_of_business")
fig, ax = plt.subplots(figsize=(7.5, 4))
styled_bar(ax, lob["Line of Business"], lob["Claim Count"], f"{name}: Claims by Line of Business", color=ACCENT)
save_fig(fig, name, "3_line_of_business")
plt.show()
if len(lob):
    results["top_lob"] = f"{lob.iloc[0]['Line of Business']} ({lob.iloc[0]['% of Claims']}%)"

# ----- 4. Cause change -> Segment change -----
print("\n--- 4. Does a Loss-Cause Change Trigger a Segment Change? ---")
ct, ct_pct, headline, trigger_rate = cause_to_segment(d)

```

```

display(headline)
print("Counts:"); display(ct)
print("Row %:"); display(ct_pct)
save_table(headline, name, "4_cause_to_segment_headline")
save_table(ct.reset_index(), name, "4_cause_to_segment_counts")
save_table(ct_pct.reset_index(), name, "4_cause_to_segment_pct")
results["trigger_rate"] = trigger_rate

# ----- 5. Contact method -----
print("\n--- 5. Contact Method Breakdown ---")
cm = breakdown_table(d, "CONTACT_METHOD", "Contact Method")
display(cm)
save_table(cm, name, "5_contact_method")
fig, ax = plt.subplots(figsize=(7.5, 4))
styled_bar(ax, cm["Contact Method"], cm["Claim Count"], f"{name}: Claims by Contact Method", color="#8172B2")
save_fig(fig, name, "5_contact_method")
plt.show()
if len(cm):
    results["top_contact_method"] = f"{cm.iloc[0]['Contact Method']} ({cm.iloc[0]['% of Claims']}%)"

# ----- Key callouts (paste-ready bullets) -----
print("\n--- KEY CALLOUTS (paste into slide notes) ---")
n_claims_val = results["n_claims"]
avg_fnol_val = results.get("avg_fnol_days", "n/a")
multi_cause_pct = cd["single_vs_multi"].iloc[1]["% of Claims"]
top_lob_val = results.get("top_lob", "n/a")
top_contact_val = results.get("top_contact_method", "n/a")

bullets = [
    f"* {name}: {n_claims_val:,} claims analyzed",
    f"* Average FNOL-to-latest-status gap: {avg_fnol_val} days",
    f"* {multi_cause_pct}% of claims had multiple loss causes",
    f"* Top line of business: {top_lob_val}",
    f"* Top contact method: {top_contact_val}",
    f"* When the loss cause changes, a segment change also happens {trigger_rate}% of the time",
]
for b in bullets:
    print(b)
print("\n")

results["summary_bullets"] = bullets
return results

```

1. Overall

[code cell 8]

```
overall_results = run_report("Overall", cuts["Overall"])
```

2. Property — Catastrophe

[code cell 9]

```
propcat_results = run_report("Property - Catastrophe", cuts["Property - Catastrophe"])
```

3. Property — Non-Catastrophe

[code cell 10]

```
propprocat_results = run_report("Property - Non-Catastrophe", cuts["Property - Non-Catastrophe"])
```

4. Non-Injury

[code cell 11]

```
noninjury_results = run_report("Non-Injury", cuts["Non-Injury"])
```

5. Cross-Cut Comparison — Overview Slide

A quick side-by-side of the 4 cuts. Good as the first "at a glance" chart in the deck.

[code cell 12]

```
all_results = [overall_results, propcat_results, propprocat_results, noninjury_results]
all_results = [r for r in all_results if r]
```

```
comparison = pd.DataFrame({
    "Cut": [r["cut"] for r in all_results],
    "Claim Count": [r["n_claims"] for r in all_results],
    "Avg FNOL Days": [r.get("avg_fnol_days", np.nan) for r in all_results],
    "Cause-Change -> Segment-Change Trigger Rate (%)": [r.get("trigger_rate", np.nan) for r in all_results],
})
display(comparison)
save_table(comparison, "cross_cut", "comparison_summary")
```

```
fig, axes = plt.subplots(1, 3, figsize=(17, 4.5))
```

```
styled_bar(axes[0], comparison["Cut"], comparison["Claim Count"], "Claim Volume by Cut", color=PRIMARY)
styled_bar(axes[1], comparison["Cut"], comparison["Avg FNOL Days"], "Avg FNOL-to-Latest-Status Days by Cut",
```

```

        ylabel="Days", color=ACCENT, value_fmt="{:,.1f}")
    styled_bar(axes[2], comparison["Cut"], comparison["Cause-Change -> Segment-Change Trigger Rate (%)"],
        "Cause-Change -> Segment-Change Trigger Rate", ylabel="%", color=GOOD, value_fmt="{:,.1f}%")

    for ax in axes:
        ax.tick_params(axis="x", labelrotation=20)

    plt.tight_layout()
    save_fig(fig, "cross_cut", "comparison_overview")
    plt.show()

    print(f"\nAll charts saved to: {os.path.abspath(CHART_DIR)}")
    print(f"All backup tables saved to: {os.path.abspath(TABLE_DIR)}")

```